GIZMO+
Product & Technical Blueprint

Music Catalog + Rights Management + Pitching CRM + Agent-Native Interface

Authoritative build specification
Version 1.0 - September 12, 2026

Purpose
This document is designed to be handed directly to a coding agent or human engineering team. It reconciles the live legacy Airtable design with a modern Turso/libSQL data architecture, a Mastra agent runtime, and a CopilotKit/AG-UI interface. It is a build specification, not a brainstorming document.

Executive Decisions

Area	Decision
Product identity	GIZMO+ is a music-specific operational control center joining catalog metadata, rights/splits, pitching CRM, outreach, and reports.
Legacy source of truth	The live GIZMO+ Airtable base is treated as the legacy product/business-logic specification, not as a database schema to copy verbatim.
Primary application database	Turso using a SQLite/libSQL-compatible relational model.
Tenancy	Recommended: one Turso workspace database per GIZMO workspace, plus a small control-plane database for workspace routing and platform metadata.
Agent runtime	Mastra: agent, tools, workflows, memory, approvals, traces/evals.
Agent/user protocol	AG-UI between the Mastra backend and the user-facing application.
Web interface	CopilotKit + GIZMO-owned React components. Chat is an operating surface for the application, not a text-only chatbot.
Dynamic UI	Use deterministic tool rendering for domain facts/actions; use A2UI selectively for compositional/dynamic surfaces.
External channels	CopilotKit Channels as a later extension of the same agent into supported messaging providers.
Files	Object storage abstraction for audio, artwork, agreements, and generated PDFs; Turso stores metadata/pointers only.
Security	Agents never receive arbitrary SQL. All reads/writes flow through typed, workspace-scoped business tools. High-impact writes require explicit confirmation where appropriate.

Critical modeling correction
A musical composition and a sound recording are separate works. ISWC belongs to the musical work; ISRC belongs to the specific recording. The legacy SONGS table mixes these concepts, so the modern model must split them while keeping the UX simple. [S17][S18][S20]

Contents

1. Product Definition and Boundaries

2. Legacy Airtable Audit

3. Canonical Domain Model

4. Turso/libSQL Architecture

5. Canonical SQL Schema

6. Business Rules and Validation

7. Mastra Agent Architecture

8. CopilotKit / AG-UI Interface Architecture

9. Conversational UX and Component Catalog

10. Traditional Application UX

11. Rights and Music Metadata Model

12. Pitching CRM and Outreach

13. Report Engine

14. Files, Media, Search, and Integrations

15. Security, Authorization, and Auditability

16. Implementation Roadmap

17. Acceptance Criteria and Test Strategy

18. Instructions for the Next Coding Agent

Appendix A. Complete Airtable Field Migration Map

Appendix B. Agent Tool Contracts

Appendix C. Sources and Research Notes

1. Product Definition and Boundaries

GIZMO+ is a music catalog, rights-management, pitching CRM, outreach, and reporting application built for people who actively manage and pitch music. It combines the structured rigor of a relational music database with an agent-native interface that can retrieve, render, validate, and act on that data.

1.1 The three core questions

- What do I own, and who owns it with me?
- Where has this song or recording been sent, to whom, and what happened?
- What should I do next?

1.2 Product pillars

Pillar	Responsibility
Catalog	Compositions, recordings, releases, artists, credits, identifiers, lyrics, metadata, assets, links.
Rights	Writer shares, publishing interests, master ownership, rights-holder identities, validation and readiness.
Pitch CRM	Industry contacts, companies, pitches, pitch items, status history, follow-ups, notes, outcomes.
Outreach	Message drafting, sending-provider abstraction, message/thread IDs, delivery/reply metadata.
Reports	Configurable metadata packages, lyrics/no-lyrics variants, rights and sync reports, PDF/export history.
Agentic UX	Natural-language operation of the same domain through Mastra + CopilotKit, with real components and safe actions.

1.3 Explicit non-goals

- GIZMO is not a DAW or audio editor.
- GIZMO is not a distributor or royalty payment platform.
- GIZMO is not a generic CRM with a music skin.
- The LLM is not the database and must never be treated as the source of truth.
- The agent must not be allowed to execute arbitrary SQL supplied or synthesized by the model.

2. Legacy Airtable Audit

The connected GIZMO+ Airtable base contains eight tables. The first five define the original GIZMO catalog and rights system. GIZMO+ adds Clients, Pitching Status, and Email, turning the system into a catalog-plus-pitching CRM.

Legacy table	Meaning	Modern treatment
Creatives	Reusable people/parties acting as artists, songwriters, publishers, master owners, contacts.	Split identity from role. Model people/organizations once, attach roles through relationships.
SONGS	Central all-in-one catalog record containing composition, recording, release, rights rollups, reports.	Split into composition, recording, release, credits, links/assets; present as one coherent Song workspace in UI.
Songwriting	Song ↔ writer junction with percentage.	composition_contributors / writer shares.
Publishing	Song ↔ publisher junction with percentage.	publishing_interests.
MasterOwner	Song ↔ master owner junction with percentage.	master_interests attached to recording.
Clients	Industry contacts with status-derived song lists.	people/organizations + contacts + roles; lists become queries.
Pitching Status	Song ↔ client relationship and state.	pitches + pitch_items + pitch_status_history/events.
Email	Pitch-linked personalized email composer/send trigger.	messages + recipients + provider metadata + explicit send workflow.

2.1 Legacy business logic worth preserving

- Enter a party once and reuse it across artist, writer, publisher, master-owner, and licensing-contact relationships.
- Ownership percentages live on relationships, not on people or songs themselves.
- Writer, publishing, and master totals are validated independently.
- Reports are assembled from canonical metadata and optional sections; they are not separate manually retyped documents.
- Pitch status is scoped to a song/recording + client relationship; the same client may have different statuses for different music.
- Email composition inherits recipient and catalog context from a pitch rather than requiring duplicate entry.

2.2 Legacy issues intentionally corrected

- One SONGS record conflates composition, recording, and release concepts.
- Client Website is typed as email and should be URL.
- UPC is numeric in Airtable but must be stored as text because identifiers may contain leading zeroes and are not arithmetic data.
- The First Name formula assumes a simple Western full-name structure; store structured names instead.
- The Send checkbox is an automation trigger masquerading as persistent state; replace it with an explicit command/status transition.
- Lookup/rollup/formula fields become joins, computed projections, constraints, or report rendering logic rather than physical domain columns.

3. Canonical Domain Model



Figure 1 - Proposed GIZMO+ canonical relational model (simplified).

3.1 Identity model: party first, role second

The Airtable Creatives table stores role-specific names on one row. The modern model should separate identity from role. A person or organization can participate in many roles without duplicating the underlying identity.

Entity	Purpose	Examples
people	Natural persons.	Songwriter, artist, producer, supervisor, licensing contact.
organizations	Legal/business entities.	Publisher, label, sync agency, management company, distributor.
party_identifiers	External identity codes.	IPI, ISNI, society-specific identifiers.
party_society_affiliations	PRO/CMO affiliation over time.	ASCAP, BMI, SESAC; extensible beyond U.S. societies.
artist_aliases	Public/stage identities that differ from legal person/org name.	Artist name, pseudonym, band name relationship.
contact_methods	Emails, phones, websites, social handles.	Multiple methods per party with primary/preferred flags.

3.2 Music model

Entity	Definition
compositions	Underlying musical works: title, lyrics, ISWC, composition copyright registration, writers/publishing.
recordings	Specific fixed recordings/masters of a composition: ISRC, duration, BPM, key, master ownership, recording credits, listen links.
releases	Commercial/release containers: album/single/EP, UPC/EAN, label, distributor, artwork, release date.
release_tracks	Ordering relationship between release and recording, including disc/track numbers.
recording_artists	Artist/featured-artist relationship to a recording.
recording_contributors	Producer, engineer, musician and other recording credits.

3.3 Rights model

Entity	Cardinality / rule
composition_contributors	Many writers per composition; share belongs to relationship.
publishing_interests	Many publishers/rightsholders per composition; share belongs to relationship.
master_interests	Many owners per recording/master; share belongs to relationship.
rights attestations	Optional evidence/agreements and confidence/verified state; one-stop readiness should be derived from validated rights plus an explicit representation attestation when needed.

3.4 CRM model

Entity	Purpose
contacts	A contactable industry relationship pointing to a person and optionally an organization.
contact_roles	Extensible roles such as music supervisor, licensing agent, label/A&R, blogger, curator, radio, sync library.
pitches	The outreach opportunity/relationship with a contact/company.
pitch_items	One or more compositions/recordings included in a pitch.
pitch_events	Immutable timeline of status changes, sends, replies, notes, meetings, outcomes.
messages	Email or other outreach messages tied to pitch; provider-neutral metadata.

4. Turso/libSQL Architecture

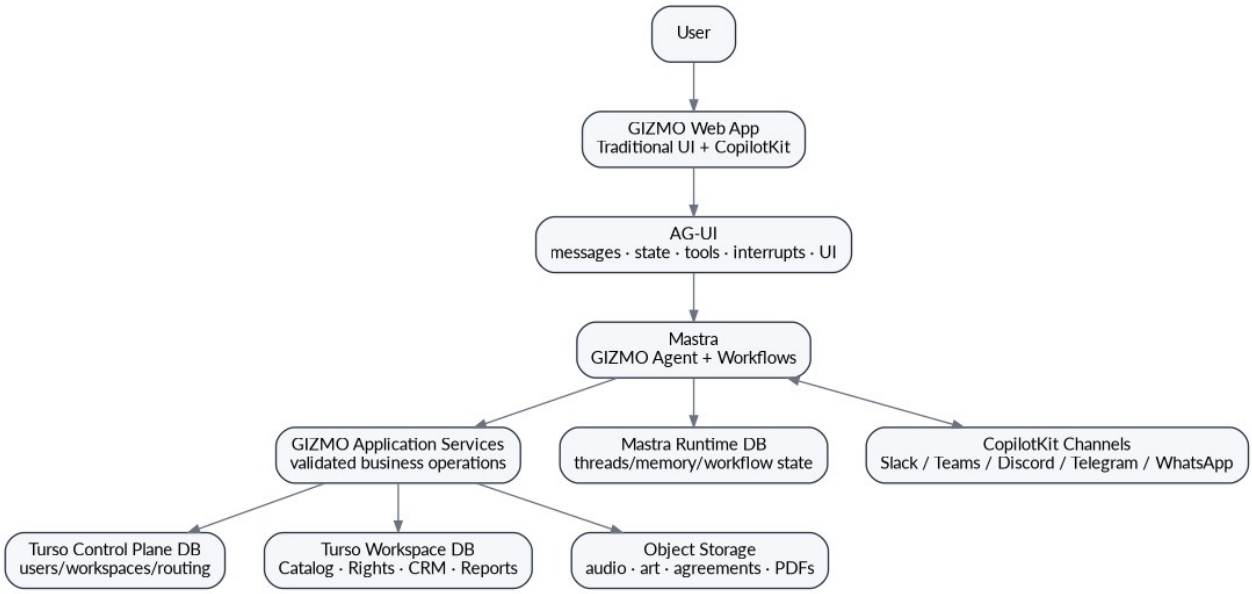


Figure 2 - Recommended application, agent, and storage boundaries.

4.1 Recommended tenancy: database per workspace

Turso explicitly supports provisioning a dedicated database per user/tenant/agent through its Platform API. For GIZMO, the tenant boundary should be the workspace, not the individual user, because a workspace may eventually contain multiple collaborators. [S1][S2]

Database	Contains	Why
gizmo-control	workspace registry, auth-user ↔ workspace membership references, database hostname/ID, plan/entitlement references, migration version, optional feature flags.	Small global control plane. Does not contain music catalog data.
gizmo-ws-<id>	All domain data for one workspace.	Hard tenant isolation, straightforward export/backup, no workspace_id required on every domain table.
gizmo-agent-runtime	Mastra operational storage: thread messages, memory, workflow/schedule state, traces where configured.	Separates AI runtime state from canonical music data. Can be split per workspace later if regulatory/isolation needs justify it.

Migration warning
Turso Multi-DB Schemas is deprecated for new users. Do not design GIZMO around automatic parent-schema propagation. Maintain numbered SQL migrations in the repository and run them against each workspace database through a controlled migration service/CLI. [S3]

4.2 Sync and offline posture

Do not make offline-first synchronization a dependency of the first web release. Current Turso documentation marks libSQL Embedded Replicas as legacy for new sync projects and recommends Turso Sync for true local-first read/write sync. Keep the domain schema SQLite-compatible so a future desktop/mobile client can adopt Turso Sync without redesigning the data model. [S4][S5]

4.3 IDs and time

- Use application-generated sortable IDs (UUIDv7 or ULID) stored as TEXT. Avoid rowid as a public identifier.
- Store timestamps as ISO-8601 UTC TEXT (or INTEGER epoch consistently) and convert at the application boundary.

- Every mutable table should have created_at and updated_at. Mutable domain records should also have created_by and updated_by when actor attribution matters.
- Use soft deletion only where recovery/history is valuable; do not apply it blindly to append-only event tables.

4.4 Database invariants

- Enable foreign-key enforcement on every connection.
- Use CHECK constraints for percentage ranges 0..100, enumerated status values where stable, and boolean INTEGER values.
- Use UNIQUE constraints for identifiers such as ISRC within a workspace when known, while allowing NULL.
- Never use floating-point equality for rights validation. Store shares as integer basis points (0..10000) or decimal text normalized in application code; this blueprint recommends basis points.

5. Canonical SQL Schema

The following is the recommended logical schema. It is intentionally SQL-first so another agent can implement it with raw libSQL or map it into the project's chosen query layer without changing the domain.

```
-- CONTROL PLANE (gizmo-control)
CREATE TABLE workspaces (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  database_name TEXT NOT NULL UNIQUE,
  database_hostname TEXT NOT NULL,
  schema_version INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE workspace_members (
  workspace_id TEXT NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  auth_user_id TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('owner','admin','member','viewer')),
  status TEXT NOT NULL DEFAULT 'active',
  created_at TEXT NOT NULL,
  PRIMARY KEY (workspace_id, auth_user_id)
);
```

5.1 Parties and contact identity

```
CREATE TABLE people (
  id TEXT PRIMARY KEY,
  given_name TEXT,
  family_name TEXT,
  legal_name TEXT NOT NULL,
  display_name TEXT NOT NULL,
  notes TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE organizations (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  organization_type TEXT,
  website_url TEXT,
  notes TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE party_identifiers (
  id TEXT PRIMARY KEY,
  party_kind TEXT NOT NULL CHECK (party_kind IN ('person','organization')),
  party_id TEXT NOT NULL,
  identifier_type TEXT NOT NULL,
  identifier_value TEXT NOT NULL,
```

```
  issuing_body TEXT,
  is_primary INTEGER NOT NULL DEFAULT 0 CHECK (is_primary IN (0,1)),
  UNIQUE (identifier_type, identifier_value)
);

CREATE TABLE party_society_affiliations (
  id TEXT PRIMARY KEY,
  party_kind TEXT NOT NULL CHECK (party_kind IN ('person','organization')),
  party_id TEXT NOT NULL,
  society_code TEXT NOT NULL,
  role TEXT,
  starts_at TEXT,
  ends_at TEXT
);

CREATE TABLE contact_methods (
  id TEXT PRIMARY KEY,
  party_kind TEXT NOT NULL CHECK (party_kind IN ('person','organization')),
  party_id TEXT NOT NULL,
  method_type TEXT NOT NULL CHECK (method_type IN ('email','phone','url','social')),
  label TEXT,
  value TEXT NOT NULL,
  is_primary INTEGER NOT NULL DEFAULT 0 CHECK (is_primary IN (0,1))
);
```

5.2 Music catalog

```
CREATE TABLE compositions (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  description TEXT,
  lyrics TEXT,
  iswc TEXT UNIQUE,
  copyright_registration_number TEXT,
  copyright_date TEXT,
  notes TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE composition_alt_titles (
  id TEXT PRIMARY KEY,
  composition_id TEXT NOT NULL REFERENCES compositions(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  title_type TEXT
);

CREATE TABLE recordings (
  id TEXT PRIMARY KEY,
  composition_id TEXT REFERENCES compositions(id) ON DELETE SET NULL,
  title TEXT NOT NULL,
  version_title TEXT,
  description TEXT,
  isrc TEXT UNIQUE,
```

```
  duration_ms INTEGER,
  bpm REAL,
  musical_key TEXT,
  preview_start_ms INTEGER,
  copyright_registration_number TEXT,
  copyright_date TEXT,
  first_release_date TEXT,
  notes TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE releases (
  id TEXT PRIMARY KEY,
  title TEXT NOT NULL,
  release_type TEXT,
  upc_ean TEXT,
  label_organization_id TEXT,
  distributor_organization_id TEXT,
  release_date TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE release_tracks (
  release_id TEXT NOT NULL REFERENCES releases(id) ON DELETE CASCADE,
  recording_id TEXT NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
  disc_number INTEGER NOT NULL DEFAULT 1,
  track_number INTEGER,
  PRIMARY KEY (release_id, recording_id)
);
```

5.3 Contributors and rights

```
CREATE TABLE composition_contributors (
  id TEXT PRIMARY KEY,
  composition_id TEXT NOT NULL REFERENCES compositions(id) ON DELETE CASCADE,
  person_id TEXT NOT NULL REFERENCES people(id),
  role TEXT NOT NULL DEFAULT 'songwriter',
  writer_share_bp INTEGER CHECK (writer_share_bp BETWEEN 0 AND 10000),
  created_at TEXT NOT NULL,
  UNIQUE (composition_id, person_id, role)
);

CREATE TABLE publishing_interests (
  id TEXT PRIMARY KEY,
  composition_id TEXT NOT NULL REFERENCES compositions(id) ON DELETE CASCADE,
  publisher_organization_id TEXT REFERENCES organizations(id),
  publisher_person_id TEXT REFERENCES people(id),
  share_bp INTEGER NOT NULL CHECK (share_bp BETWEEN 0 AND 10000),
  territory TEXT,
  agreement_asset_id TEXT,
  created_at TEXT NOT NULL,
  CHECK ((publisher_organization_id IS NOT NULL) != (publisher_person_id IS NOT NULL))
```

```
);

CREATE TABLE master_interests (
  id TEXT PRIMARY KEY,
  recording_id TEXT NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
  owner_organization_id TEXT REFERENCES organizations(id),
  owner_person_id TEXT REFERENCES people(id),
  share_bp INTEGER NOT NULL CHECK (share_bp BETWEEN 0 AND 10000),
  agreement_asset_id TEXT,
  created_at TEXT NOT NULL,
  CHECK ((owner_organization_id IS NOT NULL) != (owner_person_id IS NOT NULL))
);

CREATE TABLE recording_artists (
  id TEXT PRIMARY KEY,
  recording_id TEXT NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
  person_id TEXT REFERENCES people(id),
  organization_id TEXT REFERENCES organizations(id),
  role TEXT NOT NULL CHECK (role IN ('primary','featured')),
  billing_order INTEGER NOT NULL DEFAULT 0,
  CHECK ((person_id IS NOT NULL) != (organization_id IS NOT NULL))
);

CREATE TABLE recording_contributors (
  id TEXT PRIMARY KEY,
  recording_id TEXT NOT NULL REFERENCES recordings(id) ON DELETE CASCADE,
  person_id TEXT REFERENCES people(id),
  organization_id TEXT REFERENCES organizations(id),
  role TEXT NOT NULL,
  credit_text TEXT,
  CHECK ((person_id IS NOT NULL) != (organization_id IS NOT NULL))
);
```

5.4 CRM, pitches, messages

```
CREATE TABLE contacts (
  id TEXT PRIMARY KEY,
  person_id TEXT REFERENCES people(id),
  organization_id TEXT REFERENCES organizations(id),
  relationship_notes TEXT,
  active INTEGER NOT NULL DEFAULT 1 CHECK (active IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  CHECK (person_id IS NOT NULL OR organization_id IS NOT NULL)
);

CREATE TABLE contact_roles (
  contact_id TEXT NOT NULL REFERENCES contacts(id) ON DELETE CASCADE,
  role TEXT NOT NULL,
  PRIMARY KEY (contact_id, role)
);

CREATE TABLE pitches (
  id TEXT PRIMARY KEY,
```

```
  contact_id TEXT NOT NULL REFERENCES contacts(id),
  subject TEXT,
  current_status TEXT NOT NULL DEFAULT 'draft',
  next_follow_up_at TEXT,
  outcome TEXT,
  notes TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE pitch_items (
  id TEXT PRIMARY KEY,
  pitch_id TEXT NOT NULL REFERENCES pitches(id) ON DELETE CASCADE,
  composition_id TEXT REFERENCES compositions(id),
  recording_id TEXT REFERENCES recordings(id),
  sort_order INTEGER NOT NULL DEFAULT 0,
  CHECK (composition_id IS NOT NULL OR recording_id IS NOT NULL)
);

CREATE TABLE pitch_events (
  id TEXT PRIMARY KEY,
  pitch_id TEXT NOT NULL REFERENCES pitches(id) ON DELETE CASCADE,
  event_type TEXT NOT NULL,
  from_status TEXT,
  to_status TEXT,
  actor_type TEXT NOT NULL,
  actor_id TEXT,
  note TEXT,
  metadata_json TEXT,
  occurred_at TEXT NOT NULL
);

CREATE TABLE messages (
  id TEXT PRIMARY KEY,
  pitch_id TEXT REFERENCES pitches(id) ON DELETE SET NULL,
  channel TEXT NOT NULL DEFAULT 'email',
  direction TEXT NOT NULL CHECK (direction IN ('outbound','inbound')),
  status TEXT NOT NULL,
  subject TEXT,
  body_text TEXT,
  body_html TEXT,
  provider TEXT,
  provider_message_id TEXT,
  provider_thread_id TEXT,
  sent_at TEXT,
  received_at TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE message_recipients (
  message_id TEXT NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
  recipient_type TEXT NOT NULL CHECK (recipient_type IN ('to','cc','bcc')),
```

```
  contact_method_id TEXT REFERENCES contact_methods(id),
  address_snapshot TEXT NOT NULL,
  PRIMARY KEY (message_id, recipient_type, address_snapshot)
);
```

5.5 Assets and reports

```
CREATE TABLE assets (
  id TEXT PRIMARY KEY,
  asset_type TEXT NOT NULL,
  storage_provider TEXT NOT NULL,
  storage_key TEXT NOT NULL,
  file_name TEXT NOT NULL,
  mime_type TEXT,
  byte_size INTEGER,
  checksum TEXT,
  created_at TEXT NOT NULL,
  UNIQUE (storage_provider, storage_key)
);

CREATE TABLE entity_assets (
  asset_id TEXT NOT NULL REFERENCES assets(id) ON DELETE CASCADE,
  entity_type TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  relationship TEXT NOT NULL,
  PRIMARY KEY (asset_id, entity_type, entity_id, relationship)
);

CREATE TABLE report_templates (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  report_type TEXT NOT NULL,
  config_json TEXT NOT NULL,
  is_system INTEGER NOT NULL DEFAULT 0 CHECK (is_system IN (0,1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE generated_reports (
  id TEXT PRIMARY KEY,
  template_id TEXT REFERENCES report_templates(id),
  entity_type TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  content_text TEXT,
  content_json TEXT,
  pdf_asset_id TEXT REFERENCES assets(id),
  generated_at TEXT NOT NULL,
  generated_by TEXT
);
```

5.6 Required indexes

```
CREATE INDEX idx_recordings_composition ON recordings(composition_id);
CREATE INDEX idx_contrib_composition ON composition_contributors(composition_id);
CREATE INDEX idx_publish_composition ON publishing_interests(composition_id);
```

```
CREATE INDEX idx_master_recording ON master_interests(recording_id);
CREATE INDEX idx_pitches_contact_status ON pitches(contact_id, current_status);
CREATE INDEX idx_pitch_events_pitch_time ON pitch_events(pitch_id, occurred_at DESC);
CREATE INDEX idx_messages_pitch_time ON messages(pitch_id, created_at DESC);
CREATE INDEX idx_contact_methods_party ON contact_methods(party_kind, party_id);
```

6. Business Rules and Validation

Rule	Implementation requirement
Writer totals	For a composition, SUM(writer_share_bp) should be 10000 for a complete/validated work. Allow incomplete drafts but surface warning state.
Publishing totals	Same 10000 completion target, separate from writer total.
Master ownership	For each recording, SUM(master share) should be 10000 for complete ownership.
One-stop	Do not blindly copy Yes/No. Compute readiness from complete rights data and require an explicit representation/authorization attestation when business meaning demands it.
Composition/recording identifiers	ISWC stored only on composition; ISRC stored only on recording. [S17] [S18]
IPI	IPI identifies a natural person or legal entity with an interest in works; it belongs to a party identity, not to a song. [S19]
Pitch status	Status transitions create immutable pitch_events; current_status is a projection/cache for fast reads.
Messages	Sending is a command. A message status progresses draft → queued/sending → sent/failed; replies are inbound messages/events.
Deletion	Prevent destructive deletion of referenced canonical parties/music unless references are re-assigned or explicitly cascaded by a safe admin workflow.

6.1 Proposed pitch statuses

Keep legacy semantics but make the lifecycle explicit. Suggested starting vocabulary:

```
draft
ready
not_contacted        # legacy: Hasn't Heard
sent                 # legacy: Currently Pitching, once outreach occurs
awaiting_reply       # legacy: Awaiting Reply
follow_up_due
responded
passed               # legacy: Passed On It
interested
requested_more
accepted
closed_won
closed_lost
archived
```

The UI can present user-friendly labels while the database uses stable codes. Do not hard-code the workflow so tightly that future custom statuses become impossible; a later configurable status model can be added if user demand proves it.

7. Mastra Agent Architecture

Mastra is the backend agent/runtime. It should reason over user intent and context, but deterministic domain logic remains in GIZMO application services and typed tools. Mastra supports agents, tools, memory, workflows, approvals, observability, and durable workflows. [S6][S7][S8]



Figure 3 - Example read → render → approve → write flow.

7.1 Start with one primary agent

Use one GIZMO Agent initially. Introduce sub-agents only when a domain becomes too large or has a genuinely independent context/toolset. Premature multi-agent decomposition adds routing, context, and observability complexity.

Layer	Responsibility
GIZMO Agent	Interpret intent, resolve conversational references, select tools/workflows, explain results, request confirmation.
Tools	Small typed business capabilities: search, retrieve, validate, create/update, draft, render-ready queries.
Workflows	Repeatable multi-step processes where reliability matters more than open-ended reasoning.
Memory	Conversation continuity and user preferences; never substitute for canonical domain facts.
Application services	Transactions, authorization, validation, status transitions, report generation, provider adapters.

7.2 Tool families

Family	Representative tools
Catalog	searchMusic, getMusicWorkspace, createComposition, createRecording, updateComposition, updateRecording, addRelease, addAsset
Rights	getRightsSummary, addWriterShare, addPublishingInterest, addMasterInterest, validateRights, getReadinessIssues
Contacts	searchContacts, getContact, createContact, updateContact, listContactHistory
Pitches	searchPitches, createPitch, addPitchItem, transitionPitchStatus, addPitchNote, listFollowUps
Outreach	draftPitchMessage, createMessageDraft, sendMessage, getMessageThread
Reports	previewReport, generateReport, listReportTemplates, exportReportPdf
UI navigation	focusEntity, openWorkspacePanel, setActiveTab - frontend tools only; no domain writes

7.3 Agent vs workflow decision rule

Use agent reasoning when...	Use a Mastra workflow when...
The request is ambiguous, exploratory, or combines several possible actions.	The process is known, repeatable, and needs explicit typed steps.
The user asks a natural-language question over several domains.	Generating a pitch package follows a fixed sequence: validate → assemble → preview → approve → send.
The agent must choose among tools based on context.	Bulk follow-ups, scheduled reminders, migration/imports, or report batches require durable state and retries.

7.4 Approval policy

- Reads and non-destructive navigation: no approval.
- Low-risk draft creation: generally no approval, but render result visibly.
- Canonical writes that are easily reversible: confirmation can be contextual, not mandatory on every keystroke.

- External side effects (send email/message, share externally, delete, bulk update, overwrite ownership): require explicit human confirmation or a clear pre-authorized workflow.
- The approval UI should be an actual CopilotKit HITL/interrupt component, not a plain “Are you sure?” text message. [S14]

8. CopilotKit / AG-UI Interface Architecture

The interface requirement is not “chat next to a dashboard.” Chat is an operating surface that can retrieve canonical database facts, render application-owned components, keep UI and agent state synchronized, and invoke safe actions. AG-UI is the event protocol between the frontend and Mastra. [S9][S10]

8.1 Three rendering modes

Mode	Use it for	GIZMO examples
Deterministic tool rendering	Known domain result with a stable component and schema. Preferred default.	SongCard, RightsPanel, ContactCard, PitchTimeline, FollowUpQueue, ReportPreview.
Shared-state rendering	Ongoing collaborative state that evolves during a session.	Current pitch-building workspace, selected songs, validation checklist, report configuration.
A2UI / declarative generative UI	Composition of approved components when the exact layout is not known in advance.	Ad-hoc dashboard assembled from catalog + pitch metrics, novel comparison views.

CopilotKit can render Mastra tool calls with custom UI and supports shared state and A2UI. Deterministic tool rendering should carry factual domain results because it is easier to test, constrain, and keep visually consistent. A2UI should compose from a restricted GIZMO component catalog rather than invent arbitrary HTML. [S11][S12][S13]

8.2 Conversational reference resolution

The application needs explicit entity context. If the user opens or receives Midnight Drive and then asks “who hasn’t heard this?”, the agent should receive/maintain a selected entity reference, not infer solely from transcript text.

```
Agent/UI shared context example:
{
  "workspaceId": "ws_...",
  "focusedEntity": { "type": "recording", "id": "rec_..." },
  "selectedEntities": [ ... ],
  "activeSurface": "catalog",
  "activeFilters": { ... }
}
```

8.3 Channels are an extension, not the core web UI

CopilotKit Channels can expose an AG-UI agent into messaging providers. Current SDK documentation describes managed Slack/Teams connectivity and direct adapters for Slack, Teams, Discord, Telegram, and WhatsApp, with provider-specific capability differences. Treat Channels as a later surface over the same Mastra business tools; do not couple domain behavior to Slack/Teams semantics. [S15][S16]

9. Conversational UX and Component Catalog

9.1 Core principle

The interface rule
When the database can answer with structured domain objects, render structured domain UI. Text is for explanation, synthesis, and transitions - not as a substitute for the application.

User says	Agent/tool behavior	Rendered result
“Show me Midnight Drive.”	Resolve composition/recording → getMusicWorkspace.	MusicWorkspaceCard with artwork, player, metadata, rights readiness, pitch summary.
“Who hasn’t heard this?”	Use focused recording → query eligible/uncontacted contacts.	ContactGrid with role/company and Create Pitch actions.
“What’s wrong with the rights?”	getRightsSummary + validateRights.	RightsPanel with writer/publishing/master totals and specific gaps.
“Fix publishing to 50/50 between Jane and ACME Publishing.”	Validate parties and proposed shares → HITL → transactional update.	Editable RightsSplitEditor then success state.
“Pitch these three to Sarah.”	Create pitch draft with selected entities and contact → render preview → approve.	PitchBuilder / MessageDraftCard.
“What should I do today?”	Query follow-ups, incomplete rights, ready-to-pitch inventory.	TodayWorkspace dashboard assembled from deterministic widgets.

9.2 Initial component catalog

Component	Purpose
MusicWorkspaceCard	Summary header for a composition/recording with artwork, listen action, identifiers, readiness.
SongGrid / RecordingGrid	Browse/search results with metadata and readiness badges.
RightsPanel	Writer, publishing, master totals and holders.
RightsSplitEditor	Interactive split editing; total meter; evidence/agreements.
ReadinessChecklist	Missing identifiers, rights totals, licensing contact, listen link, assets.
ContactCard / ContactGrid	Person/company, role, methods, prior pitch history.
PitchCard	Current status, included music, contact, next follow-up.
PitchTimeline	Immutable events/messages/status transitions.
PitchBuilder	Select items, recipient, message/report package, approval.
MessageDraftCard	Editable subject/body + source facts + send confirmation.
ReportPreview	Sectioned metadata package with template toggles and export.
TodayWorkspace	Prioritized follow-ups, issues, ready items, recent activity.
SearchResults	Mixed-entity search with typed result groups.
ConflictResolver	Disambiguate “Sarah” / duplicate titles / multiple recordings.
ConfirmationCard	Human approval for external/destructive action.

10. Traditional Application UX

GIZMO must remain fully usable without conversational commands. The agent accelerates operation; it does not replace inspectable application navigation.

Primary area	Views
Home / Today	Follow-ups due, rights issues, recently edited music, ready-to-pitch, recent replies/activity.
Catalog	Compositions, recordings, releases; search/filter/gallery/table views.
People & Organizations	Artists, writers, publishers, owners, industry contacts, companies.
Rights	Validation queue, incomplete shares, agreement evidence, one-stop readiness.
Pitches	Pipeline/kanban, list, contact history, pitch detail/timeline.
Outreach	Drafts, sent messages, threads/replies, templates.
Reports	Templates, previews, generated reports, exports.
Settings	Workspace, members, integrations, email provider, storage, agent preferences.

The same React components used in deterministic chat rendering should also be reusable in full-page views. This avoids maintaining a “chat version” and an “app version” of the product model.

11. Rights and Music Metadata Model

11.1 Composition vs recording

This separation is a legal and metadata reality, not merely database normalization. The U.S. Copyright Office states that the musical composition and sound recording are distinct works. IFPI states that ISRC uniquely identifies a specific sound recording and does not identify a composition; CISAC/ISWC identifies musical works as intangible creations. [S17][S18] [S20]

Concept	Composition / musical work	Recording / master
Identifier	ISWC	ISRC
Copyright registration	PA / composition registration concept	SR / sound-recording registration concept
Primary creators	Composers, lyricists, arrangers	Performers, producer/recording contributors
Rights tracked	Writer and publishing interests	Master ownership
Can have multiple manifestations?	Yes - one work may have multiple recordings/versions.	A recording may appear on multiple releases/formats while retaining identity when it is the same recording.

11.2 IPI and societies

CISAC describes IPI as a globally unique identifier for a natural person or legal entity with an interest in an artistic work across roles such as composer, arranger, and publisher. Accordingly, GIZMO stores IPI on a party identifier record, not on a song or on role-specific duplicated name fields. [S19]

11.3 Extensibility beyond legacy fields

- PRO/CMO should be extensible; do not constrain the database to BMI/ASCAP/SESAC even though those are the legacy options.
- Allow multiple external identifiers per party and preserve issuing body/identifier type.
- Allow multiple recording links (private player, DSP, cloud file, sync link), not one Listen Link.
- Treat genres, instruments, moods, keywords, and “sounds like” as structured/extensible metadata while preserving freeform notes where useful.

12. Pitching CRM and Outreach

12.1 A pitch is a first-class domain object

A pitch is not merely a status cell. It is an auditable relationship between a contact/company and one or more pieces of music, with status, activity, messages, follow-up, notes, and outcome.

Pitch concern	Recommended model
Recipient	One primary contact per pitch initially; organization available through contact. Future multi-recipient support can be added via pitch_participants.
Music	pitch_items supports one or many compositions/recordings.
Current state	pitches.current_status for fast UI; every transition also emits pitch_event.
History	pitch_events is append-only for notes, status, sends, replies, calls, meetings, requests.
Follow-up	next_follow_up_at plus event history; a workflow can surface due items.
Campaigns	Defer dedicated campaigns until a real workflow needs bulk grouping; do not over-model MVP.

12.2 Email/provider abstraction

GIZMO should own message content, pitch linkage, recipients, and provider IDs. The provider owns actual transport and may expose delivery/open/click/reply information. Do not promise open tracking as a database capability; store provider events only when the selected integration supplies them.

- Draft generation may be agent-assisted but must cite/use canonical song and contact facts.
- Store address snapshots on outbound messages so historical records remain accurate after a contact email changes.
- Preserve provider_message_id and provider_thread_id for reply reconciliation.
- Inbound replies should append to message/thread history and produce pitch events; they should not overwrite the outbound record.

13. Report Engine

The Airtable Report formula is a significant product feature. It dynamically assembles a metadata package from optional sections. Preserve the behavior, but make templates explicit and reusable.

Legacy checkbox/section	Modern report section
Core always-present fields	Title, alt/version title, artists, BPM, key, duration, genre, description.
Rights Holders?	Writer/publisher/master sections.
Creators?	Producer, mix, mastering, musicians/credits.
Composition Details?	ISWC, composition copyright registration/date.
Recording Details?	Release, track, label, release date, ISRC, distributor, UPC/EAN, artwork credit, recording registration/date.
Metadata?	Keywords/tags, instruments, sounds-like/reference metadata.
Link To Listen?	Selected listening link + preview start.
Licensing Contact?	Licensing contact(s), representation information, one-stop readiness.
Lyrics?	Lyrics section.
ReportSansLyrics	System preset with lyrics disabled.
ReportJustLyrics	System preset containing only lyrics.

13.1 Template config example

```
{
  "name": "Sync Metadata Package",
  "sections": [
    {"id":"core","enabled":true},
    {"id":"rights","enabled":true},
    {"id":"creators","enabled":true},
    {"id":"composition","enabled":true},
    {"id":"recording","enabled":true},
    {"id":"metadata","enabled":true},
    {"id":"listen","enabled":true},
    {"id":"licensing","enabled":true},
    {"id":"lyrics","enabled":false}
  ],
  "format": "text-and-pdf"
}
```

13.2 Snapshot policy

Preview can be generated live. When a report is exported or attached to a pitch, persist a generated_reports record containing a text/JSON snapshot and optional PDF asset. This preserves what was actually sent even if catalog data changes later.

14. Files, Media, Search, and Integrations

14.1 Object storage

- Store object keys and metadata in Turso; store binary payloads in an object store.
- Use signed URLs for private audio, agreements, and generated reports.
- Separate asset identity from entity linkage so the same agreement/artwork can be referenced without duplicate uploads.
- Track checksum, MIME type, size, and original filename.

14.2 Search

MVP search can use indexed relational queries and SQLite FTS where supported by the chosen Turso/libSQL runtime. The agent should call a dedicated searchMusic/searchContacts tool rather than issue arbitrary text-to-SQL. Semantic search is optional and should only be introduced for genuinely fuzzy discovery use cases such as description/lyrics similarity.

14.3 Integrations intentionally deferred behind adapters

Capability	Boundary
Authentication	Existing auth remains authoritative. GIZMO only requires a stable auth_user_id and server-side membership check.
Billing	Existing billing remains outside music-domain DB; control-plane workspace stores entitlement reference if needed.
Email transport	Provider adapter interface; exact provider can be selected later.
Object storage	S3-compatible or other object-store adapter; exact vendor deferred.
External channel agents	CopilotKit Channels adapter; not required for core web MVP.

15. Security, Authorization, and Auditability

15.1 Request boundary

```
Authenticated request
  -> resolve auth_user_id
  -> load workspace membership from control plane
  -> resolve workspace database
  -> create request-scoped DB/service context
  -> authorize business operation
  -> run typed query/transaction
  -> return typed DTO
  -> agent/UI renders DTO
```

15.2 Agent safety rules

- No arbitrary SQL tool exposed to the model.
- No raw database credentials, Turso tokens, provider keys, or object-storage secrets in model context.
- All domain tools accept stable IDs where possible; name resolution is a separate read step that can produce a disambiguation UI.
- Every write tool re-checks workspace authorization server-side; do not trust an ID because the agent supplied it.
- Bulk/destructive/external actions require an approval mechanism and an audit event.
- Tool results should return minimal necessary data; avoid dumping entire tables into LLM context.

15.3 Audit model

Pitch events provide domain history, but administrative/canonical-data changes also benefit from an audit_log table or application event stream recording actor, action, entity, before/after summary, and timestamp. This is especially valuable for rights changes.

16. Implementation Roadmap

Phase	Scope	Exit condition
Phase 0 - Project reconnaissance	Verify existing auth, billing, routes, component system, deployment and testing conventions. Do not replace working foundation.	Architecture note + integration points.
Phase 1 - Turso control plane and migrations	Create control DB, workspace provisioning, migration runner, request-scoped workspace DB resolver.	Can provision a workspace DB, migrate it, and safely connect for an authorized user.
Phase 2 - Parties and catalog foundation	People, organizations, identifiers, contact methods, compositions, recordings, releases, credits, links/assets metadata.	Create/edit/browse a complete music record without rights/CRM.
Phase 3 - Rights	Writer, publishing, master interests; totals/readiness validation; agreements.	RightsPanel + split editor; incomplete and complete states correctly detected.
Phase 4 - Reports	Template engine, legacy-equivalent report presets, preview, text export, PDF persistence.	Full, no-lyrics, lyrics-only reports match legacy intent using normalized data.
Phase 5 - Contacts and Pitch CRM	Contacts, roles, pitches, pitch items, events, follow-up queue.	Track different statuses for different music/contact combinations with full timeline.
Phase 6 - Outreach	Message drafts, provider abstraction, send confirmation, thread/provider IDs, reply ingestion interface.	Pitch-linked messages can be drafted, approved, sent, and historically inspected.
Phase 7 - Mastra agent tools	One GIZMO Agent + typed read/write tools; memory; traces/evals; core workflows.	Natural-language queries call safe tools and return typed domain results.
Phase 8 - CopilotKit agentic UI	AG-UI, tool rendering, shared context/state, HITL components, focused-entity handling.	Chat renders real GIZMO components and can safely perform approved actions.
Phase 9 - A2UI + advanced surfaces	Restricted component catalog, selected dynamic dashboards, richer agent composition.	Dynamic surfaces remain brand-safe and schema-constrained.
Phase 10 - Channels	Expose same agent to selected external providers based on actual user demand.	Channel interactions map identity/workspace correctly and preserve provider capability boundaries.

17. Acceptance Criteria and Test Strategy

17.1 Domain tests

- Creating one composition with two recordings must preserve separate ISWC/ISRC semantics.
- A person can be songwriter on one composition, artist on a recording, and contact on a pitch without duplicate identity rows.
- Writer/publishing/master totals calculate independently and correctly at 10000 basis points.
- Same contact can have different pitch states for different recordings.
- Report presets reproduce all legacy sections from normalized data.
- Changing canonical metadata after export does not mutate a previously generated report snapshot.

17.2 Agent/tool tests

- Name ambiguity triggers disambiguation rather than an unsafe guess.
- Read-only questions never invoke write tools.
- Send/delete/bulk writes pause for explicit approval.
- Every tool rejects entity IDs from another workspace.
- Agent cannot call SQL directly.
- Tool results render the expected component and remain useful without prose.

17.3 Evals worth adding in Mastra

- Tool selection accuracy for catalog vs rights vs CRM queries.
- Entity resolution accuracy (“this song”, “Sarah”, “the second one”).
- No-hallucination eval: factual claims about catalog must be traceable to tool result.
- Write-intent safety: external side effects require approval.
- Report completeness against known fixture data.

18. Instructions for the Next Coding Agent

Primary directive
Build from this specification and the existing repository. Do not redesign the product from scratch, do not reintroduce Supabase, and do not flatten the domain back into the legacy Airtable SONGS table.

- First inspect the existing codebase and document what foundation already exists; preserve working auth, billing, and app conventions.
- Use Turso/libSQL for canonical application data. Implement database-per-workspace with a control plane unless repository constraints demonstrably require a different deployment; document any deviation before coding.
- Maintain SQL migrations in source control and a migration runner for all workspace databases. Do not use deprecated Multi-DB Schemas.
- Keep Mastra runtime storage logically separate from the GIZMO domain database.
- Implement composition and recording as separate entities even if the UI presents them as one Song workspace.
- Implement business tools/services before exposing them to the agent. Agent tools must not contain authorization logic only in prompts.
- Use deterministic CopilotKit rendering for stable domain objects. Add A2UI only after the base component catalog and typed tools exist.
- Use HITL/interrupts for external/destructive actions.
- Create fixtures that cover multi-writer, publisher, master-owner, multi-recording, multi-release, multi-pitch scenarios.
- Do not add speculative complexity that is not required by a phase exit condition. Implement phases sequentially and keep migrations/tests green.

18.1 Definition of “done” for an individual feature

- Schema/migration exists.
- Server-side authorization exists.
- Business service/tool is typed and tested.
- Traditional UI can inspect/edit the feature.
- Agent read path can retrieve and render it when applicable.
- Agent write path has correct approval behavior when applicable.
- Audit/history behavior is defined.
- Empty, loading, error, and ambiguous states are handled.

Appendix A. Complete Airtable Field Migration Map

Disposition vocabulary: Keep = direct domain concept; Normalize = move into a related canonical entity; Derived = compute/query/render rather than persist; Redesign = preserve intent but change model; Split/Move/Fix/Expand/Remove = explicit migration action.

A.1 Creatives

Legacy field	Legacy type	Disposition	Modern target / rule
Contact Name	text	Normalize	people.legal_name / organizations.name depending party type
Artist Name (if different)	text	Normalize	artist_aliases.stage_name or people.display_name
ArtistNameFormula	formula	Derived	display-name resolver; never persisted as a canonical fact
Email	email	Keep	party_emails.email
Phone	phone	Keep	party_phones.phone
Songwriter Name	text	Redesign	role comes from relationship; person name is not role-specific
Songwriter PRO	single select	Normalize	party_society_affiliations.society_code
Songwriter IPI	number	Normalize	party_identifiers(identifier_type=IPI)
Publisher Name	text	Redesign	publisher should be an organization/party, not a second name field on a person
Publisher PRO	single select	Normalize	party_society_affiliations
Publisher IPI	number	Normalize	party_identifiers
ContactInfoFormula	formula	Derived	formatted contact display
SongwriterInfo	formula	Derived	formatted credits/rights display
PublisherNameFormula	formula	Derived	party display resolver
PublisherInfo	formula	Derived	formatted publisher display
SongwritersJnct	link	Normalize	composition_contributors
PublishingJnct	link	Normalize	publishing_interests
MasterOwnerJnct	link	Normalize	master_interests
SONGJnctArtist	link	Normalize	recording_artists
SONGJnctFeaturing	link	Normalize	recording_artists(role=featured)
SONGSJnctContact	link	Normalize	composition/recording licensing contacts

A.2 SONGS

Legacy field	Legacy type	Disposition	Modern target / rule
Song Name	text	Split	compositions.title; recording.title may inherit/override
Artist	links	Normalize	recording_artists
ArtistName	lookup	Derived	join
Featuring	links	Normalize	recording_artists(role=featured)
FeaturingName	lookup	Derived	join
Alt Name	formula	Redesign	composition_alt_titles / recording.version_title
Description	text	Keep	composition.description or recording.description
Songwriters	rollup	Derived	join composition_contributors
Publishers	rollup	Derived	join publishing_interests
Master Owners	rollup	Derived	join master_interests
Album	text	Normalize	releases.title
Release Date	date	Normalize	releases.release_date / recording.first_release_date
Track Number	text	Normalize	release_tracks.track_number
Keywords	text	Redesign	tags + metadata_terms
Lyrics	text	Keep	compositions.lyrics
Instruments	text	Redesign	recording_metadata structured list
Sounds Like	text	Redesign	recording_references / freeform tags
Length	text	Normalize	recordings.duration_ms
Key	text	Keep	recordings.musical_key
BPM	number	Keep	recordings.bpm
Personal Notes	text	Keep	notes entity or private notes field
Record Label	text	Normalize	release_label / organizations
UPC Bar Code	number	Normalize	releases.upc_ean TEXT, not numeric
Producer	text	Normalize	recording_contributors(role=producer)
Mixing Engineer	text	Normalize	recording_contributors(role=mixing_engineer)
Mastering Engineer	text	Normalize	recording_contributors(role=mastering_engineer)
Musicians	text	Normalize	recording_contributors
Distributor	text	Normalize	releases.distributor_organization_id
ISRC	text	Move	recordings.isrc
ISWC	text	Move	compositions.iswc
PA Copyright # (composition)	text	Move	compositions.copyright_registration_nu

			mber
SR Copyright # (recording)	text	Move	recordings.copyright_registration_number
Copyright Date	date	Split	composition/recording copyright dates separately
Album Cover Art	attachment	Move	assets + release_assets
Art Work Artist	text	Normalize	asset_contributors / party
Listen Link	url	Redesign	recording_links table; multiple provider URLs
Preview StartAt	text	Normalize	recordings.preview_start_ms
Licensing Contact	links	Normalize	licensing_contacts / party relationships
Genre	text	Redesign	genres/tags
One Stop Ready	select	Derived	readiness assessment with optional override/attestation
PDF Agreements	attachment	Move	assets + agreements
SongwriterJnt	links	Normalize	composition_contributors
Writers Total %	rollup	Derived	SUM composition_contributors.writer_share
PublishingJnct	links	Normalize	publishing_interests
Publishing Total %	rollup	Derived	SUM publishing_interests.share
MasterOwnerJnct	links	Normalize	master_interests
Master Owner Total %	rollup	Derived	SUM master_interests.share
ContactInfoLookup	lookup	Derived	join
Report	formula	Redesign	report engine/template output
Rights Holders?	checkbox	Redesign	report template section option
Recording Details?	checkbox	Redesign	report template section option
Creators?	checkbox	Redesign	report template section option
Metadata?	checkbox	Redesign	report template section option
Lyrics?	checkbox	Redesign	report template section option
Link To Listen?	checkbox	Redesign	report template section option
Licensing Contact?	checkbox	Redesign	report template section option
Composition Details?	checkbox	Redesign	report template section option
PitchProcessJnct	links	Normalize	pitch_items
ReportSansLyrics	formula	Redesign	report preset
ReportJustLyrics	formula	Redesign	report preset

A.3 Songwriting

Legacy field	Legacy type	Disposition	Modern target / rule
RecordName	formula	Derived	display label only
Songwriter	link	Normalize	composition_contributors.party_id
Writers Percent	percent	Keep	composition_contributors.writer_share
SongwriterAndPercent	formula	Derived	formatted display
Song Name	link	Normalize	composition_contributors.composition_id
Songwriter Name	lookup	Derived	join
SongwriterInfo	lookup	Derived	join
Songwriter IPI#	lookup	Derived	join party_identifiers

A.4 Publishing

Legacy field	Legacy type	Disposition	Modern target / rule
RecordName	formula	Derived	display label
Publisher	link	Normalize	publishing_interests.publisher_party_id
Publisher Percent	percent	Keep	publishing_interests.share
PublisherAndPercent	formula	Derived	formatted display
Song Name	link	Normalize	publishing_interests.composition_id
Publisher Name	lookup	Derived	join
PublisherInfo	lookup	Derived	join
Publisher IPI#	lookup	Derived	join party_identifiers

A.5 MasterOwner

Legacy field	Legacy type	Disposition	Modern target / rule
RecordName	formula	Derived	display label
Master Owner	link	Normalize	master_interests.owner_party_id
Master Owner Percentage	percent	Keep	master_interests.share
MasterOwnerAndPercent	formula	Derived	formatted display
Song Name	link	Split	master interest belongs to recording/master, not composition
NameLookup	lookup	Derived	join

A.6 Clients

Legacy field	Legacy type	Disposition	Modern target / rule
Name	text	Normalize	contacts.display_name -> person/organization
First Name	formula	Derived	people.given_name; do not parse full names at runtime
Email	email	Keep	contact_methods
Company Name	text	Normalize	contact_organizations / organizations
Type	multi-select	Redesign	contact_roles many-to-many
Phone	phone	Keep	contact_methods
Website	email type	Fix	organizations.website_url URL
Hasn't Heard	lookup	Derived	query pitches by status
Awaiting Reply	lookup	Derived	query pitches by status
Passed On	lookup	Derived	query pitches by status
Currently Pitching	lookup	Derived	query pitches by status
Signed Agreement	attachment	Move	agreements + assets
PitchProcess	links	Normalize	pitches

A.7 Pitching Status

Legacy field	Legacy type	Disposition	Modern target / rule
PitchID	formula	Derived	stable ID + display label
SONGS	links	Redesign	pitch_items: composition/recording many-to-many
Clients	links	Normalize	pitches.contact_id
Status	single select	Expand	pitches.current_status + pitch_status_history
EmailJnct	links	Normalize	messages.pitch_id
EmailAddress	lookup	Derived	contact method
ListenLink	lookup	Derived	recording link
PreviewAt	lookup	Derived	recording.preview_start_ms
FirstName	lookup	Derived	people.given_name
RecordID	formula	Remove	DB ID already exists
Metadata	lookup	Derived	report/metadata projection

A.8 Email

Legacy field	Legacy type	Disposition	Modern target / rule
Email Record	formula	Derived	message display label
Select A Pitch	link	Normalize	messages.pitch_id
Current Status	lookup	Derived	join pitch
Sending Date	datetime	Keep	messages.sent_at
Metadata for Reference	lookup	Derived	snapshot/report reference if needed
Craft Subject Line	text	Keep	messages.subject
Craft Body of Email	text	Keep	messages.body_text/body_html
Send	checkbox	Redesign	explicit action + status machine; never a persistent boolean trigger
FirstName	lookup	Derived	contact
Email Address	lookup	Derived	recipient table
ListenLink	lookup	Derived	pitch item recording link
PreviewAt	lookup	Derived	recording
RecordID	lookup	Remove	message has own ID

Appendix B. Agent Tool Contracts

Exact runtime signatures can adapt to current Mastra APIs, but every tool should have validated inputs/outputs and a workspace-scoped service implementation.

Tool	Input	Output	Risk	Notes
searchMusic	query, filters	MusicSearchResult[]	Read	Search compositions/recordings/releases; never raw SQL.
getMusicWorkspace	entityType, entityId	MusicWorkspaceDTO	Read	Aggregate composition, recordings, releases, rights, pitch summary.
getRightsSummary	entityType, entityId	RightsSummaryDTO	Read	Returns totals, holders, readiness issues.
updateWriterShares	compositionId, shares[]	RightsSummaryDTO	Write/approval for bulk	Transaction; totals may be incomplete but validate range/duplicates.
updatePublishingShares	compositionId, shares[]	RightsSummaryDTO	Write/approval for bulk	Same.
updateMasterShares	recordingId, shares[]	RightsSummaryDTO	Write/approval for bulk	Same.
searchContacts	query, roles, historyFilters	ContactSummary[]	Read	CRM search.
getContactHistory	contactId	ContactHistoryDTO	Read	Pitches/messages/events.
createPitch	contactId, items[], notes?	PitchDTO	Write	Create draft; no external side effect.
transitionPitchStatus	pitchId, toStatus, note?	PitchDTO	Write	Append event + update current status atomically.
draftPitchMessage	pitchId, tone/options	MessageDraftDTO	Write draft	Can use LLM for prose but canonical facts come from DB.
sendMessage	messageId	SendResultDTO	External/approval	Provider adapter; idempotency key required.
previewReport	templateId, entity refs	ReportPreviewDTO	Read	Live generated output.
generateReport	templateId, entity refs, format	GeneratedReportDTO	Write	Persist snapshot / PDF asset.
getTodayWorkspace	date/filter	TodayWorkspaceDTO	Read	Follow-ups, rights issues, ready-to-pitch.

Appendix C. Sources and Research Notes

Research was checked against current vendor/standards documentation on September 12, 2026. Vendor APIs evolve; the coding agent should confirm exact package versions/signatures at implementation time without changing the architectural responsibilities defined here.

[S1] Turso Platform API - database per user/agent and programmatic database/token management
https://docs.turso.tech/api-reference/introduction

[S2] Turso Database Per Tenant overview
https://turso.tech/multi-tenancy

[S3] Turso Multi-DB Schemas (deprecated for new users)
https://docs.turso.tech/features/multi-db-schemas

[S4] Turso Embedded Replicas - legacy; Turso Sync recommended for new sync projects
https://docs.turso.tech/features/embedded-replicas/introduction

[S5] Turso Sync usage
https://docs.turso.tech/sync/usage

[S6] Mastra agent framework overview - agents, tools, memory, workflows, approvals
https://mastra.ai/ai-agent-framework

[S7] Mastra workflows - deterministic graph execution and HITL
https://mastra.ai/ai-workflows

[S8] Mastra agent memory and LibSQL storage
https://mastra.ai/ai-agents

[S9] CopilotKit Mastra integration overview
https://docs.copilotkit.ai/mastra

[S10] AG-UI protocol overview
https://docs.copilotkit.ai/ag-ui

[S11] CopilotKit tool rendering with Mastra
https://docs.copilotkit.ai/mastra/generative-ui/tool-rendering

[S12] CopilotKit shared state with Mastra
https://docs.copilotkit.ai/mastra/shared-state

[S13] CopilotKit A2UI generative UI
https://docs.copilotkit.ai/mastra/generative-ui/a2ui

[S14] CopilotKit / Mastra interrupts and human-in-the-loop
https://docs.copilotkit.ai/mastra/human-in-the-loop/interrupt-flow

[S15] CopilotKit Channels SDK
https://docs.copilotkit.ai/reference/channels

[S16] CopilotKit direct channel adapters and capability boundaries
https://docs.copilotkit.ai/reference/channels/sdk/direct-adapters

[S17] IFPI official ISRC site - recordings, not musical works
https://isrc.ifpi.org/

[S18] CISAC ISWC - musical work identifier
https://www.iswc.org/iswc

[S19] CISAC IPI - unique identification of interested parties
https://www.cisac.org/services/information-services/ipi

[S20] U.S. Copyright Office - compositions vs sound recordings
https://www.copyright.gov/register/pa-sr.html

C.1 Important current-source conclusions

- Turso explicitly documents database-per-user/agent multi-tenancy and programmatic database/token provisioning. [S1][S2]
- Turso Multi-DB Schemas is deprecated for new users; use an application migration runner instead. [S3]
- For new local-first sync work, Turso recommends Turso Sync rather than legacy Embedded Replicas. [S4][S5]
- Mastra provides agents, tools, memory, workflows, approvals and observability; its storage is runtime operational state, not the GIZMO product database. [S6][S7][S8]
- CopilotKit + Mastra uses AG-UI for user-facing state, tools and UI; tool rendering is appropriate for deterministic GIZMO components, while A2UI is suited to constrained dynamic composition. [S9]-[S14]
- CopilotKit Channels supports provider connections with managed/direct capability differences; keep it as an adapter layer over the same agent. [S15][S16]
- ISRC identifies recordings; ISWC identifies musical works; IPI identifies interested parties. [S17][S18][S19]
- The U.S. Copyright Office treats musical compositions and sound recordings as distinct works, validating the composition/recording split. [S20]

C.2 Open implementation decisions that are intentionally not blockers

Decision	Status / rule
Exact auth provider	Use existing app auth; require stable user ID and server-side membership verification.
Exact email provider	Deferred behind provider adapter.
Exact object storage vendor	Deferred behind storage adapter.
Exact ORM/query builder	Not prescribed; canonical SQL/domain is source of truth.
Offline desktop/mobile	Future; schema remains compatible with a later Turso Sync client.
Custom pitch statuses	Start with proposed fixed vocabulary; add configurability only when demanded.
Multi-agent Mastra topology	Start single-agent; split only after demonstrated context/tool pressure.

Build North Star

What GIZMO should feel like
A user should be able to browse a rigorous music-rights/CRM application normally, then switch to natural language at any moment and ask the product to find, explain, organize, validate, or act on the same canonical data. The agent should make the application easier to operate - never less trustworthy.

End of blueprint.